

Python Training Plan: 11-Day Curriculum

Course: Advanced Programming for AI (Advanced Programming using Python)

Target Audience: PGCP-AI (August 2026 batch)

Structure: 11 Days | 4 Hours of Theory per Day (Total: 44 Theory Hours)

Objective: To introduce students to Python programming, OOP principles, data science packages, database operations, web frameworks, and scraping tools in a highly structured, logical sequence.

Executive Summary of Reordering & Optimization

To maximize student comprehension and build a cohesive learning path, we have restructured the sequence of topics from the original syllabus. Here is the rationale behind the optimization:

- Structured Sequences Together (Day 2):** We grouped **Strings** (Session 3) and **Tuples** (Session 9) on Day 2. Since both are immutable sequence types, presenting them together allows students to understand Python's sequence operations and memory models in a unified way.
 - Lists as the Core Collection (Day 3):** Since **Lists** are the most versatile and widely-used collection type in Python, they are given a dedicated 4-hour block on Day 3, right after basic sequences (Strings, Tuples) and before complex associative structures.
 - Early Exception Handling (Day 4):** We moved **Exception Handling** (Session 12) to Day 4 alongside **Dictionaries**. Learning error handling early is critical; it enables students to write robust code, handle edge cases in functions, and perform safe file/database operations in subsequent days.
 - Functions & RegEx (Day 5):** **Functions and Methods** (Sessions 7 & 8) are positioned on Day 5. With control flow, data structures (Lists, Tuples, Dictionaries, Strings), and Exception Handling already covered, students are fully equipped to write clean, robust, modular functions and apply Regular Expressions.
 - Clean Transition to OOP (Day 6):** **Object-Oriented Programming** (Sessions 10 & 11) is placed on Day 6, immediately following functions. This is the natural progression from procedural/functional code to object-oriented structures.
 - Standard Tools & DBs (Day 7):** Standard Libraries, Debugging, Logging (Session 13) and Databases (Session 19) are grouped on Day 7 to solidify backend development before diving into external scientific packages.
 - Integrated Data Science Suite (Days 8-10):** We have consolidated **NumPy**, **SciPy**, **Pandas** (Sessions 14-16) and **Data Visualization** (Sessions 17-18) into a logical 3-day data analytics sequence. Day 8 introduces numerical and tabular data; Day 9 wraps up data wrangling and starts visualization; Day 10 concludes visualization.
 - Web Frameworks & Scraping (Days 10-11):** Flask/Django (Sessions 20-21) and Web Scraping (Session 22) are grouped at the end of the course to teach web service creation and consumption consecutively.
-

Daily Schedule Breakdown

Day 1: Python Introduction, IDEs & Control Flow

- **Theory Duration:** 4 Hours (Mapped to Sessions 1 & 2)
- **Core Focus:** Setting up the environment, writing basic syntax, handling user inputs, and implementing flow control.
- **Theory Topics Covered:**
 - Installing Python & introducing different IDEs (VS Code, PyCharm, Jupyter, IDLE)
 - Basic Syntax, Data Types, Variables, Operators, Input/Output operations

- Declaring variables and utilizing data types in programs
 - Flow of Control: If, If-else, Nested if-else statements
 - Looping: For loops, While loops, and Nested loops
 - Control Structures: Uses of break, continue, and pass
 - **Lab Assignments:**
 - **Q1:** Implement a program using a for loop to print the factorial (n!) of numbers from 0 to 10.
 - **Q2:** Modify the factorial program using a while loop so that it prints all factorial values that are less than 2 billion (2 billion).
-

Day 2: Sequence Types — Strings & Tuples

- **Theory Duration:** 4 Hours (Mapped to Session 3 [2T] & Session 9 [2T])
 - **Core Focus:** Mastering Python sequence types, immutability, formatting, and operations.
 - **Theory Topics Covered:**
 - **Strings:** Accessing strings, basic operations, string slicing, formatting strings, and built-in string methods.
 - **Tuples:** Introducing tuples, accessing tuples, operations on tuples, and assigning multiple values at once (tuple packing/unpacking).
 - **Lab Assignments:**
 - **Q1 (Strings):** Write a program that asks the user how many days are in a particular month, and what day of the week the month begins on (0 for Monday, 1 for Tuesday, etc.), and then prints a calendar for that month.
 - **Q2 (Strings):** Write a function to check if a user-provided sentence is a *pangram* (contains all letters of the English alphabet at least once).
 - **Q3 (Tuples):** Write a script to swap two tuples: tuple1 = (11, 22) and tuple2 = (99, 88).
-

Day 3: Mutable Sequences — Working with Lists

- **Theory Duration:** 4 Hours (Mapped to Sessions 5 & 6)
 - **Core Focus:** Comprehensive handling of lists, operations, and dynamic modifications.
 - **Theory Topics Covered:**
 - Introducing lists, defining lists, declaring, assigning, and retrieving values
 - Operations to access list elements
 - Adding, searching, and deleting list elements
 - Using list operators
 - Mapping lists (list transformations)
 - Joining lists and splitting strings (converting between lists and strings)
 - **Lab Assignments:**
 - **Q1:** Write a program to reverse a given list in Python (e.g., input [100, 200, 300, 400, 500] \$->\$ output [500, 400, 300, 200, 100]).
 - **Q2:** Write a program to find the largest and smallest numbers in a list taken as input from the user using list operations.
-

Day 4: Associative Arrays (Dictionaries) & Exception Handling

- **Theory Duration:** 4 Hours (Mapped to Session 4 [2T] & Session 12 [2T])
- **Core Focus:** Key-value data mapping and writing robust, error-tolerant Python code.
- **Theory Topics Covered:**
 - **Dictionaries:** Introducing, defining, modifying, and deleting items from dictionaries.

- **Exception Handling:** Understanding try-except blocks, handling multiple exceptions, custom exceptions, and the try-finally cleanup clause.
 - **Lab Assignments:**
 - **Q1 (Dictionaries):** Implement an encoder/decoder for the ROT-13 cipher ("rotate by 13 places") using a dictionary key map to decode: "Pnrfne pvcure? V zhpu cersre Pnrfne fnynq!".
 - **Q2 (Exceptions/Comprehensions):** Prompt the user for a list of grades separated by commas. Split the string and use list comprehension to convert them to integers. Wrap the logic in a try block to inform the user if any values cannot be converted.
 - **Q3 (Exceptions):** Investigate the behavior when a return statement is present in both the try block and the finally block of a statement.
 - **Q4 (Exceptions):** Open a file named data.txt for reading. Use a try-except block to catch the exception that arises when the file does not exist.
-

Day 5: Functions, Scopes & Regular Expressions

- **Theory Duration:** 4 Hours (Mapped to Sessions 7 & 8)
 - **Core Focus:** Procedural abstraction, reusable code components, scoping, and pattern matching.
 - **Theory Topics Covered:**
 - Defining and calling functions, types of functions, global vs. local variables (scopes)
 - Function arguments: positional, keyword, optional, and named arguments
 - Anonymous (Lambda) functions
 - Built-in helpers: type(), str(), dir(), and other built-in functions
 - Regular Expressions (RegEx) using Python's re module
 - **Lab Assignments:**
 - **Q1 (Dictionaries & Functions):** Given a student color dictionary
people={'Arham': 'Blue', 'Lisa': 'Yellow', 'Vinod': 'Purple', 'Jenny': 'Pink'}:
 1. Find out how many students are in the list.
 2. Change Lisa's favorite color.
 3. Remove 'Jenny' and her favorite color.
 4. Sort and print students and their favorite colors alphabetically by name.
 - **Q2 (String/List Functions):** Write a function translate() that translates a text into "rövarspråket" (robber's language) by doubling every consonant and placing an "o" in between (e.g., translate("this is fun") returns "tothohisosisosisofunon").
 - **Q3 (Scoping & Filtering):** Write a function filter_long_words() that takes a list of words and an integer n and returns the list of words that are longer than length n.
-

Day 6: Object-Oriented Programming (OOP) in Python

- **Theory Duration:** 4 Hours (Mapped to Sessions 10 & 11)
- **Core Focus:** Object-oriented paradigm, inheritance, polymorphism, encapsulation, and decorators.
- **Theory Topics Covered:**
 - Core OOP Concepts: Classes, Objects, and Instantiation
 - Native Data Types as classes, Declaring variables, and Referencing variables
 - Object references, Class and instance attributes
 - Python Decorators (@classmethod, @staticmethod, @property)
 - Attributes and Inheritance (single and multiple inheritance)
 - Polymorphism: Method Overloading & Overriding
 - Data hiding and encapsulation (private/protected variables)

- **Lab Assignments:**

- **Q1 (OOP Employee System):** Design an employee system using inheritance.
- Base Class: Person (attributes: id, name)
- Child Class: Employee (attributes: dept, desg; inherits from Person)
- Child Class: SalariedEmp (attributes: sal, bonus [20% of sal]; inherits from Employee)
- Formula: Net Sal = Sal + DA (10%) + HRA (15%) - PF (8%)
- Child Class: ContractEmp (attributes: hrs_worked, hourly_charges; inherits from Employee)
- Formula: Net Sal = Hours Worked * Hourly Charges
- Implement a terminal application to store employee data in a list and perform:

- a) Add new employee
 - b) Delete employee
 - c) Modify salary of employee
 - d) Search employee
 - e) Calculate Net Salary
 - f) Display All
 - g) Exit
-

Day 7: Libraries, Debugging, Logging & Databases

- **Theory Duration:** 4 Hours (Mapped to Session 13 [2T] & Session 19 [2T])
 - **Core Focus:** Practical diagnostic tooling and interacting with relational databases.
 - **Theory Topics Covered:**
 - Libraries and functional programming elements
 - Debugging basics (using PDB and IDE debuggers)
 - Logging configurations in Python (logging levels, loggers, formatters)
 - Database connections (DB-API) and working on databases using Python (SQL operations)
 - **Lab Assignments:**
 - **Q1 (Recursion):** Write a recursive function to compute the factorial of a number.
 - **Q2 (Lambdas):** Write a Lambda function to check whether a given number is an Armstrong number.
 - **Q3 (DB Integration):** Create a user table in a local database to store username, address, mobile, and email. Write a Python script to accept a username and address from a user:
 - Check if the user exists in the database.
 - If yes, display their details.
 - If not, prompt for their details and insert a new record.
-

Day 8: Data Science Foundations — NumPy & Pandas (Part 1)

- **Theory Duration:** 4 Hours (Mapped to Sessions 14, 15 & 16 [Part 1 - 4T])
- **Core Focus:** Multi-dimensional numerical arrays and tabular data representation.
- **Theory Topics Covered:**
 - Working with NumPy and SciPy: Arrays, array creation, indexing, slicing, vectorization, and mathematical operations
 - Introduction to Pandas: Series and DataFrame structures, loading tables, and basic indexing
- **Lab Assignments:**
 - **Q1 (NumPy):** Accept 20 numbers from the user and distribute them equally into 4 lists (5 numbers each). Convert these into two 2D NumPy arrays (array1 from lists 1 & 2; array2 from lists 3 & 4). Perform member-wise addition, subtraction, multiplication, and compute the exponential values of the elements in the first array.

Day 9: Data Wrangling with Pandas & Data Visualization (Part 1)

- **Theory Duration:** 4 Hours (Mapped to Sessions 14, 15 & 16 [Part 2 - 2T] & Sessions 17 & 18 [Part 1 - 2T])
- **Core Focus:** Data cleansing, grouping, transformation, and static data plotting.
- **Theory Topics Covered:**
 - Data wrangling, merging, filtering, grouping, and aggregations using Pandas
 - Data Visualization: Plotting fundamentals with Matplotlib and Seaborn
- **Lab Assignments:**
 - **Q1 (Pandas):** Read the movie users dataset from <http://bit.ly/movieusers> (pipe-separated, no headers):
 1. Add headings to the columns: sr.no, age, Gender, profession, Views.
 2. Display only the gender column.
 3. Add a new column (col6) concatenating age and gender separated by a colon (:).
 4. Group the data by age/profession, find the average views, and plot the aggregations.
 - **Q2 (Visualization):** Store product sales data for years 2010 to 2014 (5 products per year) in a list structure. Create:
 1. A stacked bar graph and a pie chart comparing yearly sales.
 2. Five separate pie charts (one for each year) representing product sales and the average sale of each product.

Day 10: Advanced Visualization & Web Frameworks (Part 1)

- **Theory Duration:** 4 Hours (Mapped to Sessions 17 & 18 [Part 2 - 2T] & Sessions 20 & 21 [Part 1 - 2T])
- **Core Focus:** Modern interactive visualization libraries and web development basics.
- **Theory Topics Covered:**
 - **Visualization:** Interactive plotting using ggplot and Plotly
 - **Web Frameworks:** Introduction to MVC/MTV architectures, comparing Flask and Django, routing, and creating basic view templates
- **Lab Assignments:**
 - **Q1 (Flask/Django):** Initialize a basic web application using Flask or Django. Create a multi-route app containing:
 - A Home page displaying a welcome message.
 - An About page rendering a static template.
 - An API route returning JSON data.

Day 11: Web Frameworks (Part 2) & Web Scraping

- **Theory Duration:** 4 Hours (Mapped to Sessions 20 & 21 [Part 2 - 2T] & Session 22 [2T])
- **Core Focus:** Dynamic web applications and retrieving data from the web.
- **Theory Topics Covered:**
 - **Web Frameworks:** Integrating databases with web frameworks, form submissions, and handling templates
 - **Web Scraping:** Working with the requests library, urllib, and building web scrapers using Scrapy
- **Lab Assignments:**
 - **Q1 (Web App):** Build out the Flask/Django application to dynamically fetch and display a list of items from a SQLite database, allowing user form submissions to append items to the database.
 - **Q2 (Web Scraping):** Create a web crawler using Scrapy (or the requests and BeautifulSoup libraries) to scrape specific data fields (such as item names, prices, or article headlines) from a public web page and save them to a file.

11-Day Theory Schedule Overview Table

Day	Theory Hours	Modules Covered	Reordering / Alignment Rationale
Day 1	4 Hours	Environment, Basic Syntax, Variables, Control Flow (If, loops, break, continue)	Direct logical starting point to establish core programming constructs.
Day 2	4 Hours	Strings & Tuples (Access, operations, slicing, formatting)	Grouped Strings and Tuples together as they are Python's primary immutable sequence structures.
Day 3	4 Hours	Lists (Creation, access, list operations, transformations, joining/splitting)	Dedicated full block to Lists, as they are the primary mutable container type.
Day 4	4 Hours	Dictionaries & Exception Handling (Try-except-finally, Custom Exceptions)	Moved Exception Handling forward. Handling errors early is a prerequisite for writing robust functions and interacting with DBs.
Day 5	4 Hours	Functions, Scopes, Arguments, Anonymous functions, Regular Expressions	Positioned after basic collections and exceptions so students can write advanced helper functions.
Day 6	4 Hours	OOP Concepts (Classes, Objects, Inheritance, Overloading, Polymorphism, Decorators)	Naturally follows procedural programming, using classes to model complex entities.
Day 7	4 Hours	Standard Libraries, Logging, Debugging, & Databases (DB-API)	Covers essential diagnostic tooling and data persistence before importing data science suites.
Day 8	4 Hours	NumPy & SciPy Foundations, Pandas Introduction (DataFrames, Series)	Commences the data science block, mapping numeric processing to tabular data frames.
Day 9	4 Hours	Pandas Advanced Wrangling, Data Visualization (Matplotlib, Seaborn)	Transition from data loading/wrangling directly to visual representations.
Day 10	4 Hours	Advanced Visualization (Plotly, ggplot), Web Frameworks Introduction (Flask/Django)	Concludes visualization and introduces web development concepts.
Day 11	4 Hours	Web Frameworks Advanced (DB integrations), Web Scraping (Requests, urllib, Scrapy)	Completes web frameworks and transitions directly into scraping data from external websites.

Total Theory Hours: 44 Hours

Total Lab Assignments: 24 exercises structured into 11 distinct lab sessions corresponding to daily theory.